WORDPRESS DAY AI

# Boas práticas de desenvolvimento seguro com Inteligência Artificial no WordPress

**Lucas Catão de Moraes**
Fundador & CEO, DolutechAI

IA para Desenvolvimento

Faro · 24.10.2026 · 15:30

QUEM VOS FALA

# Lucas Catão de Moraes



**Fundador & CEO da DolutechAI**
Dedicação exclusiva à startup de cibersegurança e IA

**SOC AI Agent autónomo**
A plataforma da DolutechAI para PME europeias

**Autor de cibersegurança**
Mais de 4 milhões de downloads das obras publicadas

**Fundador do Blog Dolutech**
Tecnologia e cibersegurança para a comunidade

**Segurança WordPress e Linux**
Hardening, plugins e infraestrutura de servidores

**Pós-graduação em IA e Machine Learning**
Desenvolvimento e investigação em Reinforcement Learning

O PONTO DE PARTIDA

# Compilar não é o mesmo que estar seguro

**≈100%**

do código gerado pelos modelos tem sintaxe válida: corre, compila, “funciona”

**56%**

passa nos testes de segurança, em média. Quase metade das amostras falha

***A IA otimiza para o pedido que fazemos, não para a ameaça que não descrevemos.***

Fonte: Veracode, GenAI Code Security Report 2026 (julho de 2026), 11 modelos avaliados

AGENDA

# Cinco frentes em 30 minutos

| | | |
|---|---|---|
| 01 | **Os erros clássicos que a IA repete** | Nonces, capabilities, sanitização, escape, SQL e REST |
| 02 | **Revisão humana, skills e CI/CD** | Regras para o assistente e portões no pipeline |
| 03 | **Dados sensíveis, cofres e APIs** | Prompts, segredos e chaves que não podem sair |
| 04 | **Dependências sugeridas pela IA** | Pacotes inventados e cadeia de fornecimento |
| 05 | **Agentes, MCP e política de IA** | Menor privilégio, MCP seguro e regras da equipa |

01 · ERROS CLÁSSICOS

# Seis falhas que a IA repete no WordPress

### Nonce ausente

Pedidos que alteram dados sem verificação: abre a porta a CSRF

### Capability ausente

Sem current_user_can(), qualquer utilizador autenticado executa a ação

### Entrada sem sanitização

$_POST e $_GET usados tal como chegam

### Saída sem escape

echo direto de dados: XSS armazenado ou refletido

### SQL concatenado

Variáveis dentro da query em vez de $wpdb→prepare()

### REST aberta

permission_callback com __return_true em rotas que expõem dados

# Pedi à IA: “guarda a bio do utilizador via AJAX”

Gerado pela IA

```php
add_action( 'wp_ajax_update_bio', 'dt_update_bio' );

function dt_update_bio() {                    // 1 2
    global $wpdb;
    $id  = $_POST['user_id'];                 // 3
    $bio = $_POST['bio'];                     // 3
    $wpdb->query( "UPDATE {$wpdb->usermeta}
        SET meta_value = '$bio'
        WHERE user_id = $id
        AND meta_key = 'description'" ); // 4
    echo 'Bio atualizada: ' . $bio;           // 5
    wp_die();
}
```

1. **Sem nonce**
   CSRF: uma página externa faz o pedido em nome da vítima
2. **Sem verificação de permissões**
   Um subscritor altera a bio de qualquer utilizador
3. **Entrada sem sanitização**
   $_POST usado tal como chega
4. **SQL concatenado**
   SQL injection na tabela de utilizadores
5. **Saída sem escape**
   XSS refletido na resposta

# A mesma função, escrita como deve ser

Corrigido

```
add_action( 'wp_ajax_update_bio', 'dt_update_bio' );

function dt_update_bio() {
    check_ajax_referer( 'dt_update_bio', 'nonce' ); // 1

    $user_id = absint( $_POST['user_id'] ?? 0 );
    if ( ! current_user_can( 'edit_user', $user_id ) ) { // 2
        wp_send_json_error( 'Sem permissão.', 403 );
    }
    $bio = sanitize_textarea_field(                  // 3
        wp_unslash( $_POST['bio'] ?? '' ) );
    update_user_meta( $user_id, 'description', $bio ); // 4

    wp_send_json_success(
        array( 'bio' => esc_html( $bio ) ) );        // 5
}
```

1. **check_ajax_referer()**
   Valida o nonce enviado pelo JavaScript
2. **current_user_can( 'edit_user', $id )**
   Permissão sobre este utilizador em concreto
3. **wp_unslash() + sanitize_*()**
   Limpa a entrada à chegada
4. **API nativa em vez de SQL**
   update_user_meta() trata da query
5. **esc_html() na saída**
   Escapar no momento em que se imprime

# REST API: a linha que expõe tudo

**Gerado pela IA**

```
'permission_callback'
    ⇒ '__return_true',
```

## Consequência

Qualquer visitante anónimo consulta a rota. Num endpoint de encomendas, isso significa dados pessoais de clientes expostos, com implicações de RGPD.

## O que mudou

Permissão explícita, argumentos sanitizados e query preparada com placeholder %d.

**Corrigido**

```
register_rest_route( 'dt/v1', '/orders/(?P<id>\d+)', array(
    'methods'             ⇒ WP_REST_Server::READABLE,
    'callback'            ⇒ 'dt_get_order',
    'permission_callback' ⇒ function () {
        return current_user_can( 'manage_woocommerce' );
    },
    'args' ⇒ array(
        'id' ⇒ array( 'sanitize_callback' ⇒ 'absint' ),
    ),
) );

function dt_get_order( WP_REST_Request $request ) {
    global $wpdb;
    return $wpdb→get_row( $wpdb→prepare(
        "SELECT * FROM {$wpdb→prefix}dt_orders WHERE id = %d",
        $request['id'] ) );
}
```

# Trate o código da IA como o PR de um júnior




## Sniffs do WordPress Coding Standards que apanham os erros do bloco 01

- `WordPress.Security.NonceVerification`
- `WordPress.Security.ValidatedSanitizedInput`
- `WordPress.Security.EscapeOutput`
- `WordPress.DB.PreparedSQL`

# Peça segurança de forma explícita

**Prompt de sistema / regras do projeto**

```
Contexto: plugin WordPress, PHP 8.1+, WP 6.9+.

Requisitos de segurança obrigatórios:
- verificar nonce em pedidos que alteram dados
- verificar current_user_can() com a capability
  mínima necessária
- sanitizar toda a entrada, escapar toda a saída
- usar $wpdb→prepare() ou APIs nativas,
  nunca SQL concatenado
- REST: permission_callback explícito;
  __return_true só em rotas públicas
- não adicionar dependências sem as justificar

No fim, lista os riscos que ficaram por tratar.
```

## O prompt ajuda. Não garante.

- O modelo pode ignorar requisitos em pedidos longos
- Não conhece o modelo de ameaça do seu projeto
- Pode afirmar que aplicou uma proteção que não está no código
- **A revisão continua obrigatória**

# Skills: ensine as regras ao assistente uma vez

.claude/skills/wp-security-review/SKILL.md

```
---
name: wp-security-review
description: Rever código PHP de plugins e temas
  WordPress. Usar sempre que houver AJAX, REST,
  formulários, SQL ou upload de ficheiros.
---
# Revisão de segurança WordPress
1. Alterar dados: nonce + current_user_can()
2. Entrada: wp_unslash() + sanitize_*()
3. Saída: esc_html(), esc_attr(), wp_kses()
4. SQL: $wpdb→prepare() ou APIs nativas
5. REST: permission_callback; __return_true só se pública
6. Correr: vendor/bin/phpcs --standard=WordPress
7. Rever a própria resposta e corrigir (RCI)
8. Listar os riscos que ficaram por tratar
```

**AGENTS.md**
Contexto e regras do projeto num ficheiro que vários agentes leem

**Skills (SKILL.md)**
Formato aberto: Claude Code, Codex, Cursor e outros carregam a skill quando a tarefa pede

**WordPress/agent-skills**
18 skills oficiais: plugins, REST API, Abilities API, PHPStan, WP-CLI

**Guia OpenSSF (2025)**
Instruções de segurança para assistentes de código e a técnica RCI

# CI/CD: portões que o código da IA tem de passar

1. **Revisão humana obrigatória**
   Branch protection e CODEOWNERS
2. **PHPCS + WPCS e PHPStan**
   Build falha em erros de segurança
3. **SAST**
   Semgrep, Snyk Code, CodeQL ou SonarQube
4. **Secret scanning**
   Push protection, gitleaks ou TruffleHog
5. **Dependências (SCA)**
   Snyk Open Source, Dependabot, composer/npm audit
6. **Revisão de segurança por IA**
   Segunda opinião (ex.: /security-review)

**GitHub Actions com permissões mínimas**

```
on: pull_request
permissions: { contents: read }   # GITHUB_TOKEN mínimo
jobs:
  seguranca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<sha>   # fixada por SHA
      - run: composer install --no-scripts
      - run: vendor/bin/phpcs --standard=WordPress
      - run: pipx run semgrep scan --config p/php --error
      - uses: snyk/actions/php@<sha>
        env:
          SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
```

**O pipeline também é alvo**

Março de 2025: tags da tj-actions/changed-files reapontadas para código que expunha segredos do CI.

# O que cola num prompt sai do seu ambiente

## Nunca cole

- wp-config.php: credenciais da base de dados, chaves e salts
- Application Passwords, tokens e chaves de API
- Dumps da base de dados de produção
- Logs com emails, IPs ou dados de clientes
- Código de clientes sem autorização contratual

## Em vez disso

- Dados fictícios e fixtures para reproduzir o problema
- Segredos substituídos por placeholders e variáveis de ambiente
- Ferramentas com política de retenção e de treino conhecida
- Modelos locais quando o projeto ou o cliente o exigem
- Ficheiros sensíveis excluídos do contexto do agente

**Colar dados pessoais num prompt é um tratamento de dados. O RGPD aplica-se.**

# Segredos: num cofre, fora do código e do contexto

### Cofre

Vault / OpenBao, Infisical, 1Password, Doppler ou o secret manager da cloud

### Injeção em runtime

Variáveis de ambiente ou referências (ex.: op://) resolvidas só na execução

### Aplicação e CI

O segredo vive em memória: não em ficheiros, commits, logs ou prompts

**Claude Code · .claude/settings.json**

```
{
  "permissions": {
    "deny": [
      "Read(./.env)", "Read(./.env.*)",
      "Read(./wp-config.php)",
      "Read(./secrets/**)"
    ]
  }
}
```

### Higiene de segredos

- O mesmo bloqueio noutras ferramentas (.cursorignore e equivalentes)
- Âmbito mínimo e validade curta para cada chave
- Rotação imediata após exposição em prompt ou log
- Secret scanning ativo e .env fora do Git

# Chaves de API: só no servidor, nunca no browser

**Erro comum**

```
wp_localize_script( 'dt-chat', 'DT', array(
    'apiKey' ⇒ get_option( 'dt_api_key' ),
) );
```

Qualquer visitante vê a chave no código-fonte da página.

**WordPress 7.0 · Connectors**

1. **Variável de ambiente** ex.: ANTHROPIC_API_KEY, injetada pelo cofre
2. **Constante PHP** define() no wp-config.php
3. **Base de dados** guardada sem cifra, só mascarada na interface

## Boas práticas

1. **O browser chama a sua rota REST**
   Com nonce, permission_callback e limite de pedidos
2. **Minimizar o que sai do site**
   Só o texto necessário: sem emails, IPs ou IDs de clientes
3. **Uma chave por site e ambiente**
   Com limite de gastos no fornecedor e rotação
4. **Logs e retenção sob controlo**
   Não registar prompts com dados pessoais; preferir retenção mínima

# Slopsquatting: quando a IA inventa a dependência

**19,7%**

dos pacotes sugeridos pelos modelos não existiam

**205 474**

nomes de pacotes inventados, todos diferentes

**A IA sugere um pacote que não existe**
composer require ou npm install de um nome plausível

**O atacante regista esse nome**
Os mesmos nomes inventados repetem-se entre pedidos

**O programador instala sem verificar**
Copiar e colar o comando sugerido

**Código malicioso corre no build ou no servidor**
Scripts de instalação, autoload, credenciais expostas

Fonte: Spracklen et al., “We Have a Package for You!”, 576 mil amostras de código, 16 modelos (arXiv 2406.10279)

# Quatro controlos antes de cada nova dependência

01

## Confirme que existe

Registo oficial (Packagist, npm), repositório público, autor identificável e histórico

02

## Avalie a manutenção

Data do último release, issues abertas, utilização real e vulnerabilidades conhecidas

03

## Fixe as versões

composer.lock e package-lock.json no repositório, instalação reproduzível

04

## Audite no pipeline

composer audit e npm audit no CI, inventário de dependências (SBOM)

**A dependência mais segura é a que não adiciona.** Antes de instalar, veja se o WordPress já tem uma API.

# Quando a IA deixa de sugerir e passa a agir no site

**Conteúdo não confiável**
comentários, posts, formulários, emails

↓ *prompt injection*

**Agente de IA**
lê o conteúdo e decide que ferramenta chamar

↓

**Ferramentas**
MCP, Abilities API, REST API

↓

**WordPress**
dados, utilizadores, encomendas

## Guardrails de conceção

1. **Utilizador dedicado com a role mínima**
   Nunca a conta de administrador
2. **Application Password própria e revogável**
   Uma por agente, com nome e data
3. **Só leitura por defeito**
   Escrita apenas nas abilities que precisam
4. **Confirmação humana nas ações destrutivas**
   Apagar, publicar, reembolsar, alterar utilizadores
5. **Staging primeiro e registo de todas as ações**
   Para auditar tudo o que o agente fez

# MCP: cada servidor é código com permissões

**OWASP MCP Top 10 (beta)**

| | |
|---|---|
| MCP01 | Gestão de tokens e exposição de segredos |
| MCP02 | Escalada de privilégios por excesso de âmbito |
| MCP03 | Tool poisoning: descrições envenenadas |
| MCP04 | Ataques à cadeia de fornecimento |
| MCP05 | Injeção e execução de comandos |
| MCP06 | Prompt injection via conteúdo do contexto |
| MCP07 | Autenticação e autorização insuficientes |
| MCP08 | Falta de auditoria e telemetria |
| MCP09 | Servidores MCP sombra, não aprovados |
| MCP10 | Injeção de contexto e partilha excessiva |

## Ao instalar um servidor MCP

- Fonte verificada e versão fixada
- Local via stdio sempre que possível
- Tokens emitidos para esse servidor, sem passthrough
- Reaprovar quando as ferramentas mudam

## Ao expor o WordPress (MCP Adapter)

- Utilizador dedicado com capabilities limitadas
- meta mcp.public só nas abilities necessárias
- HTTP público apenas com abilities de leitura
- Application Passwords ou OAuth, com logs de uso

# Uma ability com o mínimo de privilégio

**WordPress 6.9+ · Abilities API**

```php
add_action( 'wp_abilities_api_init', function () {
    wp_register_ability( 'dolutech/listar-encomendas', array(
        'label'       ⇒ __( 'Listar encomendas', 'dt' ),
        'description' ⇒ __( 'Encomendas recentes.', 'dt' ),
        'category'    ⇒ 'loja',
        'input_schema' ⇒ array(                          // 1
            'type'       ⇒ 'object',
            'properties' ⇒ array( 'limite' ⇒ array(
                'type' ⇒ 'integer', 'maximum' ⇒ 20 ) ),
        ),
        'output_schema' ⇒ array( 'type' ⇒ 'array' ),
        'execute_callback'    ⇒ 'dt_listar_encomendas',
        'permission_callback' ⇒ function () {            // 2
            return current_user_can(
                'view_woocommerce_reports' );
        },
    ) );
} );
```

1. **input_schema limita o pedido**
   No máximo 20 registos por pedido do agente
2. **permission_callback mínima**
   Capability de leitura, não de gestão da loja
3. **Uma ability, uma tarefa**
   Leitura e escrita em abilities separadas
4. **REST só quando necessário**
   show_in_rest apenas nas abilities que o cliente usa

# Política de IA da equipa: cabe numa página

### Ferramentas aprovadas

Lista fechada e contas empresariais; código de clientes nunca em contas pessoais

### Classificação de dados

O que pode ir para cada ferramenta: público, interno, confidencial, pessoal

### Revisão obrigatória

Todo o código de IA passa por PR revisto; quem aprova responde por ele

### Transparência

Uso relevante de IA indicado no PR e comunicado ao cliente quando o contrato o exige

### Agentes e MCP

Inventário de agentes, servidores MCP e permissões; nada entra sem aprovação

### Formação e incidentes

Literacia em IA para a equipa e plano para segredos ou dados expostos

Referências: AI Act da UE (art. 4.º, literacia em IA) · ISO/IEC 42001 · NIST AI RMF · OWASP Top 10 for Agentic Applications 2026

PARA SEGUNDA-FEIRA

# Checklist de desenvolvimento seguro com IA

- [x] Rever todo o código da IA como um pull request
- [x] Nonce e current_user_can() em todas as ações que alteram dados
- [x] Sanitizar à entrada, escapar à saída
- [x] $wpdb→prepare() ou APIs nativas, nunca SQL concatenado
- [x] permission_callback real em todas as rotas REST
- [x] Skill de segurança e política de IA partilhadas pela equipa
- [x] CI com PHPCS, SAST, secret scanning e auditoria de dependências
- [x] Actions fixadas por SHA e GITHUB_TOKEN só de leitura
- [x] Segredos num cofre; nada de segredos ou dados pessoais em prompts
- [x] Chaves de API só no servidor, nunca no browser
- [x] Confirmar que cada pacote sugerido existe e é mantido
- [x] MCP e agentes com utilizador dedicado, permissões mínimas e logs

WORDPRESS DAY AI

# Obrigado

# Perguntas?

**Lucas Catão de Moraes**
Fundador & CEO, DolutechAI · dolutech.ai

IA para Desenvolvimento